

CGA NLU 活用ガイド

対象：ボット・対話設計運営者、AI/NLU 専門担当者

このドキュメントは、CGA の **ML**、**Semantic**、**LLM** エンジンを選択し、各エンジンに合ったデータ・設定・テスト・品質改善方法をまとめます。

1. NLU 基本概念

- 意図：ユーザーの発話が何を要求するかを示す分類単位
- 学習文：意図を学習したり、検索基準として使用するために登録したユーザー表現
- オブジェクト：発話から抽出する必要がある業務値または名前
- 辞書：ドメイン用語・同義語・ユーザー表現を解釈するための資産
- しきい値：結果が利用可能であると認める最小基準
- 類似度：入力と候補データが意味的にどれだけ近いかを示す値
- 回答エンジン：分類結果に応じて決まった回答または検索・生成回答を選択する領域

2. エンジン選択基準

区分 ML	Semantic	LLM	
基本的な方法意図と学習 文章中心分類	埋め込みとベクトル検索 の中心	ベクターイラスト	CLIPARTO LLM モデルと ディレクティブベースの 処理
準備データ意図、学習文 、オブジェクト、辞書	意図または検索知識、埋 め込み・Vector DB 意図 、指令、Provider・モデ ル		
適切な出発点区別可能な 業務意図があり、学習文 章を管理するとき表現が			

違ってても意味のある検索 が必要なとき LLM ベース の解析・生成とモデル運 用が必要なとき			
主な検証精度、誤分類、 文章バランス	検索類似度、しきい値、 インデックス状態応答の 一貫性、指令の遵守、遅 延・費用		
主なリスク意図の重複、 データの不均衡埋め込み 互換性、インデックスの 不一致モデル変更、プロ ンプトの影響、応答偏差			

詳細モデルとサポートステータスは、ボット作成画面で選択可能な項目に基づいて確認します。

3. 共通運用手順

1. ボットとバージョンを確認してください。
2. NLU 方式とモデルを確認します。
3. 回答方法を確認してください。
4. データまたは接続設定を準備します。
5. 学習・インデックス生成・モデル適用タスクを実行します。
6. シミュレータで代表発話をテストします。
7. 分析・評価・対話履歴で結果を確認します。
8. 原因を分類し、データを修正して再テストします。

4. 品質チェック共通原則

- 1つの学習文には1つの重要な意図が含まれています。

- 同様の意図には、互いに区別できる名詞・動詞・状況が含まれます。
- 特定の意図だけにデータが過度に追い込まれないようにバランスを確認してください。
- ドメイン用語は、オブジェクトまたは辞書として管理するかどうかを最初に決定します。
- テスト発話は、学習文と同じ表現だけを繰り返すことはありません。
- 分析結果で誤分類が繰り返される意図の対を優先的に改善します。

4.2 学習文の作成規則

注 NLU ガイドの原則を CGA 運用データに適用する場合は、次の手順を使用します。

1. 意図名を業務目的が明らかにするように設定します。
2. 1 つの学習文には 1 つの重要な要求のみを入れます。
3. 同じ意図を異なる語順・口述・表現で用意します。
4. 他の意図と重なる単語だけを増やすのではなく、意図を区別する状況と行動を含めます。
5. 同じまたはほとんど同じ文を繰り返し登録しません。
6. 特定の意図にのみ文章が集まらないか、意図別に比較します。

たとえば、**休暇残日数の照会**と**休暇申請方法**を分割するときは、両方の意図に**休暇**という単語を入れるよりも、**残り日数**と**申請手順**のように区別基準があります。

4.3 QA・知識データ作成規則

QA や文書ベースの知識を準備するときは、質問と回答の範囲を明確にします。

- 質問は、実際のユーザーが入力できる表現で作成されます。
- 回答は質問に直接対応するように書かれており、1 つの回答に複数のトピックを混在させません。
。
- 文書ベースのデータは、タイトル・本文の区切りが維持されるように原文構造を整理します。
- テーブルまたはリストが重要な場合は、変換後に意味が維持されていることを確認してください。
- 文書の改訂時に、既存の文書と新しい文書が同時に検索されないかどうかを確認します。

4.1 運用チェック記録

エンジン設定や学習データを変更するときは、以下の項目をまとめて記録します。

- ボットとバージョン
- 変更前・後 NLU 方式、モデル、回答方式
- 変更した意図・個体・事前・QA または接続設定
- 実行した学習・インデックス付け・適用作業と画面状態
- 代表成功発火と失敗発火
- 変更前・後シミュレータ・分析・評価結果

この記録がないと、エンジン変更の影響とデータ変更の影響を分離するのは困難です。

5. エンジン別ガイド

- ML エンジン活用
- Semantic エンジン活用
- LLM エンジン活用

6. ML エンジンの活用

6.1 設定

現在の画面で ML モデルとして確認される項目は、DeepLearning Lite、TF-IDF Linear、Keyword Baseline です。実際の学習接続状態は、選択可能状態とバージョン設定を一緒に確認します。

6.2 データの作成

- 意図別に代表表現と様々な表現を用意します。
- 1 つの文に 1 つの意図のみを含めます。
- メソッドの問い合わせやエラーの問い合わせなど、区別が必要な意図は表現を混在させません。
- 意図別の文数と表現の多様性をまとめて確認します。

6.3 テストと改善

1. 代表発話と境界発話をそれぞれ用意します。
2. シミュレータで結果を確認します。
3. 分析・評価画面で誤分類意図と繰り返される表現を探します。
4. 競合意図の違いが明らかになるようにデータを修正します。
5. 再学習し、同じテストを繰り返します。

6.4 注意事項

学習文の数を増やすだけで品質が自動的に改善されるとは想定していません。単純な母語の変化よりも意図を区別するコア表現と実際のユーザーのさまざまな表現を追加します。

ML 変更前のチェック：

1. 競争意図と区別される基準を一文で説明します。
2. 各意図に代表表現・変形表現・境界表現があることを確認します。
3. ある文に複数の意図を混ぜた学習文がないことを確認します。
4. 変更前の成功 発話が変更後も保持されるか再テストします。

7. Semantic エンジンの活用

7.1 タイプ

- **Semantic - Vector Worker:** CGA Vector Worker 基本モデルと Local Vector DB を使用するタイプ
- **Semantic - External Embedding:** 外部埋め込みと Local Vector DB を接続するタイプ

7.2 設定

Semantic NLU を選択すると、Intent Vector DB 接続設定が表示されます。External Embedding タイプでは、検索 API アドレスと API Key 選択入力、Index 名を使用できます。デフォルトの Vector Worker タイプは、デフォルトの接続とインデックス名を解決します。

7.3 モデル選択

現在のコードには、韓国語一般文書用 **ko-sroberta**、多言語・表・書式用 **multilingual-e5**、長い文書・約款用 **bge-m3** など外部埋め込み選択肢が定義されています。実際の運用選択は、文書の特性、運用接続、埋め込み互換性を一緒に確認します。

7.4 テストと改善

1. 代表的な質問と表現を準備します。
2. 意図データまたは知識データが Vector DB に反映されていることを確認してください。
3. シミュレータでは、意味が同じ表現と異なる表現をそれぞれテストします。
4. 検索結果、類似度、しきい値、インデックスの状態を確認します。
5. 検索が合わない場合、データ・埋め込み・インデックスの組み合わせを一緒にチェックします。

実際のインデックス作成完了状態と検索結果画面は、ブラウザ・実行検証後にこの手順を確定する必要があります。

Semantic 変更前のチェック：

1. 埋め込みモデルと検索対象データが互換性があることを確認します。
2. Index 名と接続先が現在のボット・バージョンと一致することを確認します。
3. 外部検索 API を使用している場合は、応答仕様と認証設定を担当者に確認してください。
4. インデックスが更新される前の結果は、新しいデータの品質として解釈されません。

8. LLM エンジンの活用

8.1 設定

LLM Engine を選択すると、LLM Provider および Provider 固有の詳細モデルを設定できます。画面で確認された Provider の選択肢は、Gemini、ChatGPT、Claude、Groq、Cerebras、Mistral、Ollama、OpenRouter です。たとえば、ChatGPT を選択すると、**GPT-4o mini（標準）**と **GPT-4o（高品質）**が表示されます。Provider とモデルのリストは操作設定によって異なり、Ollama を使用している場合は別のアドレス入力項目が表示されることがあります。

8.2 ディレクティブと答え方

LLM は、NLU モデルの選択と回答方法の選択を一緒に確認する必要があります。

- LLM Engine の回答：LLM が回答を生成する方法
- LLM Engine RAG の回答：検索された知識と LLM を一緒に使用する方法
- 決まった回答：事前定義された回答の使い方

ディレクティブには、言葉、回答形式、制限事項を明確に記録します。ディレクティブの変更後、同じテストセットで一貫性と例外応答を比較します。

8.3 テストと改善

1. 代表的な質問、あいまいな質問、禁止または例外の質問を準備します。
2. Provider と詳細モデルを固定します。
3. 同じ入力を繰り返して応答の一貫性を確認します。
4. ディレクティブの遵守と回答の根拠を確認してください。
5. 遅延・費用・失敗応答を記録します。

LLM 変更前のチェック：

1. Provider と詳細モデルを変更前・後に記録します。
2. 同じ入力・指示文・回答方式を使って比較します。
3. 現実性、フォーマットコンプライアンス、禁止応答、失敗応答を別々に確認します。
4. 遅延やコストが重要な業務であれば、品質結果とともに記録します。

9. 分析と品質改善

分析画面では、累積分類結果と適用されたステップを確認します。現在の画面に表示される分類ステップには、除外/無視、スモールトーク、Exacting Matching、ルール、ML、セマンティック、LLM などが含まれます。

品質改善手順：

1. 失敗した発話を集めます。

2. 実際に適用された分類ステップを確認してください。
3. 意図・検索・指示・回答のうちどの領域の問題かを分離します。
4. 最小範囲のデータを修正します。
5. 既存の成功発話と失敗発話を一緒に再テストします。

9.1 テストセットの設定

エンジンを変更したりデータを変更したりするときは、1つのテスト発話しか使用しません。

テストセット	目的	例示基準
代表発話通常の主な使用経路の確認 よく使われる表現		
変形発火表現の変化に対する処理の 確認語順、話し方、浮き書きの変化		
境界発火競争意図と区別されること を確認同様の単語を含む他の要求		
例外発話未支援・あいまいな要請の 処理確認意図に属さない質問		
回帰発火変更前の成功結果の維持確 認以前に成功した発話		

変更前と変更後に同じセットを使用して、エンジン変更やデータ補完の効果を比較することができます。テスト結果はボット・バージョン・エンジン・モデル・回答方式とともに記録します。

10. エラー対応

症状優先確認	次のアクション
予想意図に分類されていません。ボット・バージョン ・エンジン・データ状態	代表・境界発火と競争意図を共に検討

Semantic 検索結果がありません。Vector DB、Index、埋め込み互換性接続・インデックス・データ反映状態確認	
LLM 回答が揺れます。Provider、モデル、ディレクティブ、入力	同じテストセットで比較し、ディレクティブを絞り込む
学習または準備状態が完了していません。学習履歴、エラーメッセージ、選択の組み合わせ画面の状態を記録し、運営担当者に伝えます。	

検証用 ML ボットで確認されたケース：

- 症状：学習要求はキューに登録されていますが、更新後に**未学習**のままになる
- シミュレータ結果: **意図未分類, 学習文がないため意図分類を実行できません。**
- 判断：学習文の保存成功と ML 学習完了は別々の状態なので、学習完了前に分類品質を評価しない
- 処置: 学習履歴の完了状態を確認し、履歴がなければ運営担当者にボット・バージョン・エンジン・要求時刻を伝達

DB や CLI を直接修正して状態を合わせません。画面のボット・バージョン・エンジン・エラーメッセージ・発生時刻を記録して運営担当者に伝達します。

11. 学習・索引の実行と状態判定

- 実行前にボット UUID、バージョン、言語、エンジン、モデル、回答方式、保存済み資産、基準テストセットを確認します。
- ML 学習と Semantic 索引は 3 分以上かかる場合があります。要求メッセージだけでは成功と判断しません。
- 学習履歴で最終状態を確認してからボットテストを行います。履歴なし、未学習のまま、空の索引、LLM 呼び出し失敗は失敗として扱います。

12. Score と Cut-off の運用

1. 現在の Cut-off と類似度基準を記録します。
2. 正解、誤答、境界、非対応発話の Score 分布を収集します。
3. 一度に一つの基準だけを変更し、同じ回帰セットを再実行します。
4. 誤受理と誤拒否の両方を評価し、表示パーセントだけを最適化しません。

13. 多言語 NLU 運用

CGA の UI とボット言語は、韓国語、英語、簡体字中国語、日本語、ベトナム語、フランス語、ドイツ語に対応します。識別子と API 契約は固定し、発話、メッセージ、エンティティ、評価セットは対象言語で作成します。

メッセージは要求言語を優先し、なければボット言語へフォールバックします。形態、空白、敬語、アクセント、言語固有の表現を個別に検証し、直訳だけに依存しません。

14. 制御されたエンジン実験

- 運用バージョンを上書きせず、別の作業バージョンを作成します。
- テストセットを固定し、データ、閾値、モデル、Provider、Prompt、索引設定のうち一つだけを変更します。
- 精度、失敗種類、回答一貫性、遅延、費用、運用制約を記録します。
- 合意基準を満たし、既存成功例を維持したバージョンだけを運用候補にします。

15. 品質改善の例

配送問い合わせでは、配送予定と現在状態を分離し、代表・変形・境界・例外・回帰発話を準備し、注文番号抽出を確認します。再学習または再索引後に全セットを比較し、誤りが減っても既存成功発話の Score やIntentが悪化していないことを確認します。

16. 運用反映チェックリスト

- [] ボット UUID、バージョン、言語、エンジン、モデル、回答方式を記録した。

- [] 同じテストセットで変更前後を比較した。
- [] 学習履歴で学習または索引成功を確認した。
- [] Score、類似インテント衝突、回帰発話を確認した。
- [] 対象言語のメッセージとエンティティ抽出を確認した。
- [] 実チャネル、対話、API、Queue 履歴を確認した。

関連ドキュメント

- CGA マニュアル全体表示
- CGA Getting Started
- CGA ユーザーマニュアル
- エンジン比較表